Introduction au live coding

Arts numériques, VJing et scène strasbourgeoise

Support de cours — TOPLAP Strasbourg

Table des matières

1 Qu'est-ce que le live coding ? 3

1.1 Définition . . . 3
1.2 Ce qui distingue le live coding . . . 3
1.3 TOPLAP et le manifeste . . . 3
1.4 Une généalogie courte . . . 4
1.5 Le mouvement Algorave . . . 4
1.6 Ce que le live coding n'est pas . . . 4

2 Pourquoi le live coding aux arts numériques ? 4

2.1 Le code comme matériau, pas comme outil . . . 5
2.2 Filiation historique . . . 5
2.2.1 La musique algorithmique . . . 5
2.2.2 L'art génératif visuel . . . 5
2.2.3 Sol LeWitt et l'instruction comme œuvre . . . 6
2.2.4 La démoscène . . . 6
2.2.5 Glitch art, net art et culture du dysfonctionnement . . . 6
2.3 Le code comme partition . . . 6
2.4 Dimension politique : code visible, code partagé . . . 6
2.5 Repères bibliographiques . . . 7

3 Cartographie des outils du cursus 7

3.1 Sonic Pi . . . 7
3.2 Strudel . . . 8
3.3 SuperCollider . . . 10
3.4 Hydra . . . 11
3.5 Resolume . . . 13
3.6 Vue d'ensemble . . . 14

4 Live coding et VJing : deux gestes pour l'image en direct 14

4.1 Brève histoire du VJing . . . 14
4.2 Deux gestes différents . . . 15
4.3 Ce que les deux ont en commun . . . 15
4.4 Zones de recouvrement et hybrides . . . 15
4.4.1 Hydra utilisé comme outil VJ . . . 15
4.4.2 TouchDesigner et le node-based . . . 16
4.4.3 Routage entre mondes . . . 16
4.4.4 Live coding dans Resolume ? . . . 16

4.5 Audio-réactivité : techniques communes 16
4.5.1 Analyse FFT 16
4.5.2 OSC (Open Sound Control) 16
4.5.3 MIDI 17
4.6 Synthèse 17

1 Qu'est-ce que le live coding ?

1.1 Définition

Le **live coding** est une pratique artistique qui consiste à écrire et modifier du code informatique **en temps réel devant un public**, pour produire du son, de l'image, ou les deux simultanément. Le code n'est pas préparé puis exécuté lors d'un concert : il est tapé, transformé, cassé et reconstruit pendant la performance elle-même.

C'est un déplacement important : **le code devient l'instrument**, et l'acte d'écrire devient l'acte de jouer. Là où un guitariste manipule des cordes, le live codeur manipule des fonctions, des boucles, des paramètres.

1.2 Ce qui distingue le live coding

Trois éléments le différencient des pratiques électroniques voisines :

— **Par rapport au DJ** : le live codeur ne mixe pas des œuvres existantes, il *génère* le matériau. Le son ou l'image n'existe pas avant la performance.
— **Par rapport au musicien « laptop » classique** : il ne déclenche pas un set figé préparé en studio. Tout — ou presque — est ouvert à la modification.
— **Par rapport à la programmation classique** : pas de séparation entre écrire et exécuter. Le langage est conçu pour que toute modification soit prise en compte immédiatement, sans interruption du flux sonore ou visuel. C'est le principe du *hot-swapping*.

Cette dernière contrainte technique est fondamentale : un environnement de live coding est, avant tout, un environnement où **on peut changer le programme pendant qu'il tourne**.

1.3 TOPLAP et le manifeste

En février 2004, à Hambourg, un groupe d'artistes-chercheurs (Alex McLean, Click Nilson, Julian Rohrhuber, entre autres) fondent **TOPLAP** — *Temporary Organisation for the Promotion of Live Algorithm Programming*. Ils rédigent un manifeste qui pose les principes encore référents aujourd'hui :

> *« Give us access to the performer's mind, to the whole human instrument. »*
> *« Obscurantism is dangerous. Show us your screens. »*
> *« Programs are instruments that can change themselves. »*
> *« Algorithms are thoughts. Chainsaws are tools. Chainsaws are dangerous. »*

Deux exigences à retenir :

1. **« Show us your screens »** — montrer le code projeté est devenu une signature esthétique et politique de la pratique. Le public voit la pensée à l'œuvre. Ce qui est en jeu : la transparence, le refus de la magie noire, la dimension pédagogique de la performance.
2. **L'erreur fait partie du jeu** — un crash, une boucle infinie, un son saturé : ces accidents sont assumés. Cela rapproche le live coding de l'improvisation jazz plus que du concert classique.

1.4 Une généalogie courte

Le live coding ne sort pas de nulle part. Il s'inscrit dans plusieurs lignées :

— **La musique algorithmique** (Xenakis, Koenig, puis les premiers travaux sur ordinateur dans les années 1950–1960) : composer en formalisant des règles plutôt qu'en écrivant des notes.
— **Les langages de synthèse sonore** : Music-N de Max Mathews (années 1950), puis CSound, plus tard SuperCollider (1996, James McCartney) — ce dernier sera décisif car il introduit une architecture client/serveur qui rend le hot-swapping naturel.
— **La démoscène** : ces compétitions de programmation graphique en temps contraint ont popularisé l'idée d'un code performé, esthétique en soi.
— **L'improvisation libre et le free jazz** : pour le rapport au temps, à l'écoute, à l'erreur.

Le tournant est l'apparition de **TidalCycles** (Alex McLean, 2009) pour la musique, puis d'environnements visuels comme **Hydra** (Olivia Jack, 2018) qui démocratisent radicalement la pratique en la mettant dans un navigateur.

1.5 Le mouvement Algorave

En 2012, Alex McLean et Nick Collins inventent le terme **Algorave** : des soirées où des live codeurs jouent de la musique dansante générée par algorithme, le code projeté derrière eux. C'est aujourd'hui le format de diffusion principal de la scène, avec des éditions partout dans le monde.

L'Algorave a deux conséquences importantes :

— Le live coding sort des conférences académiques et des galeries pour entrer dans des contextes festifs.
— Il revendique une dimension **populaire** : on peut danser sur du code, et le code peut être laid, brut, inachevé.

1.6 Ce que le live coding n'est pas

Pour clarifier dès le début, quelques confusions fréquentes à lever :

— Ce n'est pas du **VJing** : on y reviendra (Resolume), mais le VJ assemble des clips et des effets via une interface, là où le live codeur écrit des instructions textuelles.
— Ce n'est pas de la **musique assistée par ordinateur** classique (Ableton, Logic) : pas de timeline, pas de clips préparés, pas de mixage en aval.
— Ce n'est pas réservé aux informaticiens : la plupart des outils du cursus (Sonic Pi, Strudel) sont conçus pour entrer par la musique, pas par le code.

2 Pourquoi le live coding aux arts numériques ?

Le live coding n'est pas une bizarrerie née d'un bricolage de geeks. C'est l'aboutissement contemporain d'une lignée artistique longue, qui traverse la musique, les arts visuels, la performance et la pensée du code. Cette section trace cette filiation pour situer la pratique : un live codeur ne joue pas *contre* l'histoire de l'art, il la prolonge avec un nouvel instrument.

2.1 Le code comme matériau, pas comme outil

Première bascule conceptuelle : aux arts numériques, le code n'est pas un outil au service d'une œuvre extérieure (un script qui rendrait une image), il *est* la matière de l'œuvre. Le geste artistique consiste à manipuler des symboles, des règles, des structures formelles — pas seulement à appuyer sur un bouton qui les exécute.

Cette bascule a un nom dans le champ académique : les **software studies**, portées notamment par Lev Manovich (*The Language of New Media*, 2001) et Wendy Hui Kyong Chun (*Programmed Visions*, 2011). Le code est étudié comme un fait culturel, pas seulement technique.

2.2 Filiation historique

2.2.1 La musique algorithmique

Bien avant les ordinateurs, des compositeurs ont cherché à formaliser la composition par des règles. **Iannis Xenakis** (1922–2001) écrit dans les années 1950 des œuvres stochastiques — *Pithoprakta*, *Achorripsis* — où les nuées de notes sont calculées à partir de distributions de probabilités. Sa *Musique formelle* (1963) est un texte fondateur : la composition y est explicitement décrite comme un programme.

Gottfried Michael Koenig, à Utrecht puis à Cologne, développe dès les années 1960 des programmes de composition (*Project 1*, *Project 2*). **John Cage** et l'usage du I Ching, **Karlheinz Stockhausen** et la sérialisation intégrale, **Pierre Boulez** et ses structures formelles : tous explorent l'idée d'une partition comme *ensemble d'instructions* plutôt que comme suite de notes.

Le live codeur hérite directement de cette tradition. Quand on écrit en Strudel `n("0 3 7 12").scale("minor")`, on déclare une règle, pas un événement.

2.2.2 L'art génératif visuel

Parallèlement, dès les années 1960, des artistes utilisent des ordinateurs pour générer des images :

- **Vera Molnár** (1924–2023), franco-hongroise, pionnière de l'art génératif. Ses séries de carrés perturbés (*(Dés)Ordres*, années 1970) explorent la déformation systématique d'une grille par des règles paramétrables.
- **Frieder Nake** et **Georg Nees**, Allemagne, exposent dès 1965 des œuvres faites par traceurs.
- **Manfred Mohr**, qui transforme l'hypercube en motif graphique récurrent.
- **Roman Verostko** et le mouvement des *algorists*, défini par Jean-Pierre Hébert avec une formule programmatique :

  > *« Si l'algorithme avait une part dans votre œuvre, alors vous étiez un algorist. »*

Ces artistes pratiquent un live coding *différé* : ils écrivent un programme, le lancent, observent, modifient. Le live codeur compresse cette boucle dans le temps de la performance.

2.2.3 Sol LeWitt et l'instruction comme œuvre

En dehors du numérique, l'art conceptuel a posé une question parallèle. Sol LeWitt, dans ses *Wall Drawings* (à partir de 1968), définit une œuvre comme une instruction écrite que des assistants exécutent — l'œuvre *est* l'instruction. Sa formule de 1967 :

> *« The idea becomes a machine that makes the art. »*

Le parallèle avec le live coding est frappant : la performance live coding rend visible cette « machine de l'idée » pendant qu'elle tourne.

2.2.4 La démoscène

À partir des années 1980, sur Commodore 64, Amiga, puis PC, des collectifs (les *demogroups*) produisent des démos : courtes performances audiovisuelles temps réel, calculées par le programme lui-même, pas pré-rendues. Compétitions célèbres : *Assembly* en Finlande, *Revision* en Allemagne. Catégories « 4k » ou « 64k » : produire une œuvre audiovisuelle de plusieurs minutes en moins de 64 kilo-octets de code.

La démoscène apporte trois choses au live coding :
— L'esthétique du code performé, où le code lui-même peut être beau.
— La culture du *shader* GLSL en temps réel (cf. Hydra, partie 3).
— L'idée que la contrainte technique est une matière esthétique.

2.2.5 Glitch art, net art et culture du dysfonctionnement

Dans les années 1990–2000, des artistes comme **JODI** (Joan Heemskerk et Dirk Paesmans), **Cory Arcangel**, ou **Rosa Menkman** (*Glitch Studies Manifesto*, 2010) revendiquent l'erreur, le bug, l'incompréhension du code par lui-même comme matière artistique. Le live coding accueille cet héritage : un crash en performance n'est pas une honte, c'est un événement.

2.3 Le code comme partition

Une métaphore travaille toute la pratique : **le programme est une partition**. Cette métaphore a des conséquences concrètes dans le design des langages dédiés :

— En **TidalCycles** et **Strudel**, la *mini-notation* décrit des cycles temporels comme une partition (cf. partie 3). `"bd sd [bd cp] sd"` se lit littéralement comme une mesure rythmique.
— En **SuperCollider**, les `Pbind` sont des suites d'événements paramétrables — une partition événementielle.
— En **Hydra**, la chaîne d'opérateurs est une partition de transformations visuelles : `osc().rotate().out()` se lit de gauche à droite comme une mesure.

Le musicien ou la musicienne retrouve donc, à un niveau plus abstrait, la logique d'une portée : on déclare un *matériau* et un *traitement temporel*.

2.4 Dimension politique : code visible, code partagé

Le live coding est traversé par une éthique — héritée de la culture du logiciel libre, du hacker, et du DIY :

— **Code visible.** « Show us your screens » (TOPLAP) refuse l'effet boîte noire des laptops dont on ne sait pas s'ils jouent ou s'ils lisent un fichier.
— **Code libre.** La quasi-totalité des outils du cursus (Sonic Pi, Strudel, SuperCollider, Hydra) sont open source. Resolume est l'exception, et c'est volontaire dans ce parcours : il représente l'industrie du VJ « pro ».
— **Code partageable.** Les sessions sont souvent diffusées intégralement (le `.scd`, le `.tidal`) : la performance est aussi un objet pédagogique.
— **Communautés horizontales.** Algoraves, Toplap Slack, forums TidalCycles, serveurs Discord : la transmission est faite pair à pair plus que par des canaux institutionnels.

C'est pourquoi initier quelqu'un au live coding, ce n'est pas seulement enseigner des outils — c'est l'introduire à une **culture**.

2.5 Repères bibliographiques

Pour aller plus loin, quelques textes structurants :

— Geoff Cox & Alex McLean, *Speaking Code : Coding as Aesthetic and Political Expression*, MIT Press, 2013. La référence pour penser le code comme parole/performance.
— Nick Collins & Alex McLean (dir.), *The Oxford Handbook of Algorithmic Music*, Oxford University Press, 2018. Très complet, avec un chapitre dédié au live coding.
— Iannis Xenakis, *Musique formelle*, 1963 ; rééd. Stock, 1981.
— Lev Manovich, *The Language of New Media*, MIT Press, 2001.
— Florian Cramer, *Words Made Flesh : Code, Culture, Imagination*, 2005, disponible librement en ligne.
— Wendy Hui Kyong Chun, *Programmed Visions : Software and Memory*, MIT Press, 2011.
— Casey Reas & Chandler McWilliams, *Form+Code in Design, Art, and Architecture*, Princeton Architectural Press, 2010.
— Rosa Menkman, *The Glitch Moment(um)*, Institute of Network Cultures, 2011.

Et deux conférences fondatrices à voir en captation :

— Andrew Sorensen, *The Concert Programmer* (TEDx Queensland, 2014).
— Sam Aaron, *Programming as Performance* (multiples versions, dont SuperCollider Symposium 2013).

3 Cartographie des outils du cursus

Le cursus mobilise cinq outils, couvrant trois familles : les environnements pédagogiques (Sonic Pi, Strudel), les environnements professionnels de synthèse (SuperCollider), le live coding visuel (Hydra), et le VJing logiciel (Resolume). Chacun a un *paradigme* — une manière de penser le temps, le matériau et la performance — qu'il faut comprendre avant la syntaxe.

3.1 Sonic Pi

Auteur : Sam Aaron, université de Cambridge, 2012.
Plateforme : Linux, macOS, Windows, Raspberry Pi.
Langage : dérivé de Ruby, fortement domain-specific.
Sous le capot : le serveur SuperCollider (`scsynth`).

Paradigme

Sonic Pi est conçu comme un *instrument pédagogique d'abord, instrument de scène ensuite*. La syntaxe est verbale, lisible, on peut la lire à voix haute (`play 60` = joue le do médian). Le concept central est la `live_loop` : une boucle nommée qui peut être réécrite à chaud pendant qu'elle tourne.

Exemple minimal

```
live_loop :kick do
  sample :bd_haus
  sleep 0.5
end

live_loop :melody do
  use_synth :prophet
  play choose([:c4, :e4, :g4, :a4]), release: 0.3
  sleep 0.25
end
```

Deux boucles tournent en parallèle. Modifier le `choose` ou le `sleep`, recharger : la nouvelle version remplace l'ancienne au prochain cycle, sans coupure.

Forces

Courbe d'apprentissage très douce. Excellent pour l'éducation musicale (utilisé dans des écoles primaires britanniques). Communauté pédagogique active.

Limites

Moins puissant que SuperCollider en synthèse. Pas de patterns aussi riches que TidalCycles/Strudel. Le rendu sonore par défaut est « daté ».

Ressources

Site officiel : `https://sonic-pi.net/`. Tutoriel intégré, en français pour partie. Livre : Sam Aaron, *The Magic of Code*, à paraître.

3.2 Strudel

Auteur : Felix Roos, à partir de 2022.
Plateforme : navigateur web (aucune installation).
Langage : JavaScript avec une mini-notation textuelle inspirée de TidalCycles.
Filiation : portage JavaScript de **TidalCycles** (Alex McLean, 2009), lui-même écrit en Haskell.

Paradigme

Strudel pense la musique comme **des cycles**. Un cycle dure par défaut une seconde. Tout pattern est une fonction du temps qui découpe ce cycle en événements.

C'est le *paradigme fonctionnel pur* appliqué à la musique : pas d'état mutable, pas de boucle visible. On compose des fonctions qui transforment des patterns. Cette approche — venue de Tidal — est sans doute la plus puissante du live coding musical contemporain.

La mini-notation

Le cœur de Strudel est sa *mini-notation*, un mini-langage texte décrivant des patterns rythmico-mélodiques :

```
"bd sd [bd cp] sd"
```

Lecture : un cycle est divisé en 4 temps. Aux temps 1, 2, 4 : kick, snare, snare. Au temps 3, deux subdivisions imbriquées : kick puis clap.

Quelques opérateurs essentiels :

```
"a b c"        // séquence
"a [b c]"      // subdivision
"a*3"          // répétition (a a a)
"a/2"          // étirement (a sur deux cycles)
"a@2 b"        // pondération de durée
"a(3,8)"       // rythme euclidien : 3 pulsations sur 8 pas
"a | b | c"    // alternance (random sur cycles)
"<a b c>"      // alternance déterministe sur cycles
```

Rythmes euclidiens

L'opérateur `(k,n)` génère un **rythme euclidien** : *distribuer k pulsations le plus régulièrement possible sur n pas*. Algorithme de Bjorklund, appliqué aux rythmes par Godfried Toussaint en 2005 (*The Euclidean Algorithm Generates Traditional Musical Rhythms*). Étonnamment, il produit des rythmes traditionnels du monde entier :

```
(2,5)  // x . x . .           : khafif-e-ramal (Perse)
(3,8)  // x . . x . . x .     : tresillo (Cuba)
(5,8)  // x . x x . x x .     : cinquillo
(7,16) // x . . x . x . x . . x . x . x .  : Samba
```

C'est un exemple typique de la *matière mathématique* qui irrigue le live coding.

Exemple complet

```
stack(
  s("bd(3,8)"),
  s("hh*8").gain(0.4),
  n("0 3 5 7 12".add("<0 -3 2>"))
    .scale("D:minor")
    .s("sawtooth")
    .lpf(sine.range(400, 2000).slow(4))
).cpm(120)
```

Trois couches superposées : kick euclidien, charley, ligne de basse en mineur naturel transposée par cycles, avec un filtre passe-bas modulé par un sinus lent.

Forces

Web, donc rien à installer. Très expressif. Communauté très active. Lien direct avec la pensée Tidal/Haskell.

Limites

Sons par défaut limités (samples basiques). Pour la synthèse fine, mieux vaut router vers SuperCollider via OSC ou utiliser les synthés WebAudio fournis.

Ressources

`https://strudel.cc/`. Workshop interactif intégré (*Learn*). Documentation : `https://strudel.cc/learn/`. Pour la théorie sous-jacente : Alex McLean, *Making Programming Languages to Dance to : Live Coding with Tidal* (FARM 2014).

3.3 SuperCollider

Auteur : James McCartney, première version 1996, libre depuis 2002.
Plateforme : Linux, macOS, Windows.
Langage : `sclang`, langage à objets inspiré de Smalltalk.
Architecture : client (`sclang`) / serveur audio (`scsynth`) via OSC.

Paradigme

SuperCollider est l'environnement *adulte* de la synthèse temps réel. C'est le moteur qu'utilisent Sonic Pi (en interne) et TidalCycles (par OSC). On y descend pour :

— Définir ses propres synthés (`SynthDef`).
— Faire de la synthèse fine : FM, granulaire, soustractive, additive, physique.
— Travailler avec des structures de patterns sophistiquées (`Pbind`, `Pmono`, `Pdef`).

Exemple : un SynthDef

```
SynthDef(\bass, { |freq=110, amp=0.3, gate=1, cutoff=800|
    var env, sig;
    env = EnvGen.kr(Env.adsr(0.01, 0.2, 0.6, 0.3), gate, doneAction: 2);
    sig = Saw.ar(freq) + Saw.ar(freq * 1.005);
    sig = RLPF.ar(sig, cutoff, 0.3);
    sig = sig * env * amp;
    Out.ar(0, sig ! 2);
}).add;
```

Un synthé soustractif : deux scies légèrement désaccordées passées dans un filtre passe-bas résonant, modulées par une enveloppe ADSR.

Exemple : un Pbind

```
Pdef(\groove,
    Pbind(
        \instrument, \bass,
        \scale, Scale.minor,
        \degree, Pseq([0, 2, 4, 7, 4, 2], inf),
```

```
        \octave, 3,
        \dur, Pseq([0.25, 0.25, 0.5, 0.25, 0.25, 0.5], inf),
        \cutoff, Pwhite(400, 2000)
    )
).play;
```

Un pattern joue le SynthDef défini plus haut, en mineur, avec un cutoff aléatoire à chaque note.

Forces

Puissance maximale en synthèse. Stabilité (en production depuis 1996). Vraie portabilité du même `.scd` entre OS. Communauté de chercheurs / compositeurs (Berlin, Goldsmiths, IRCAM).

Limites

Courbe d'apprentissage raide. Documentation parfois aride. La syntaxe sclang dépayse au début.

Ressources

`https://supercollider.github.io/`. Livres : Wilson, Cottle & Collins, *The SuperCollider Book*, MIT Press, 2011 ; Eli Fieldsteel, *SuperCollider for the Creative Musician*, 2024 (et sa chaîne YouTube — ressource pédagogique majeure).

3.4 Hydra

Auteur : Olivia Jack, 2018.
Plateforme : navigateur web (aucune installation).
Langage : JavaScript, paradigme de chaînage de fonctions.
Sous le capot : compile vers du **GLSL fragment shader**, exécuté par WebGL sur le GPU.

Paradigme

Hydra pense l'image comme un **réseau de buffers vidéo** qui se modulent les uns les autres. Inspiré directement des *video synthesizers* analogiques (Sandin, Rutt-Etra, années 1970), où des oscillateurs et des LFOs étaient patchés en temps réel.

Quatre sortes d'objets :

- **Sources** : `osc`, `noise`, `shape`, `voronoi`, `gradient`, `solid`, `src(o0)` pour boucler une sortie sur elle-même.
- **Géométrie** : `rotate`, `scale`, `kaleid`, `pixelate`, `repeat`.
- **Couleur** : `color`, `invert`, `contrast`, `luma`, `posterize`.
- **Combinaisons** : `add`, `mult`, `blend`, `diff`, `modulate`, `modulateScale`, `modulateRotate`.

Quatre buffers de sortie : `o0`, `o1`, `o2`, `o3`. Le buffer `o0` est rendu par défaut.

Exemple minimal

```
osc(20, 0.1, 1.2)
  .rotate(0.3)
  .kaleid(5)
```

```
  .modulate(noise(3, 0.2))
  .out()
```

Un oscillateur sinusoïdal coloré, tourné, kaléidoscopé, dont les coordonnées UV sont déformées par un bruit. Quatre lignes pour une vidéo musicale entière.

Audio-réactivité

Hydra expose `a.fft[0]`, `a.fft[1]`, etc. : niveaux par bande spectrale du signal d'entrée (microphone ou onglet audio).

```
a.show()                  // visualise les bandes (réglage)
a.setBins(4)              // découpe en 4 bandes
a.setSmooth(0.8)          // lissage temporel

osc(10, 0.1, 1)
  .modulate(noise(3))
  .scale(() => 1 + a.fft[0] * 2)   // graves font respirer l'image
  .rotate(() => time * 0.1)
  .out()
```

Le `() => ...` est crucial : c'est une fonction réévaluée à chaque frame. Sans elle, la valeur serait figée à l'instant du chargement.

Mathématique sous-jacente

Chaque pixel est calculé indépendamment sur le GPU. La chaîne Hydra est traduite en un fragment shader GLSL où chaque coordonnée `(u,v)` produit une couleur `(r,g,b,a)`. Par exemple, `osc(freq, sync, color)` produit approximativement, en GLSL :

```
vec4 osc(vec2 uv, float freq, float sync, float color) {
    float r = sin(uv.x * freq + time * sync) * 0.5 + 0.5;
    float g = sin(uv.x * freq + time * sync + color * 2.0) * 0.5 + 0.5;
    float b = sin(uv.x * freq + time * sync + color * 4.0) * 0.5 + 0.5;
    return vec4(r, g, b, 1.0);
}
```

Comprendre cette traduction est utile : un live codeur Hydra avancé peut injecter directement du GLSL via `setFunction` pour étendre le langage.

Forces

Aucune installation. Esthétique riche obtenue très vite. Couplage audio simple. Permet de glisser vers le GLSL pur si on veut creuser. Communauté très active.

Limites

Tout est en mémoire GPU dans le navigateur : peu d'options pour intégrer des vidéos lourdes ou de la 3D complexe. Pas de support facile de NDI/Spout en sortie sans astuces (extension `Atom-Hydra`, capture d'écran via OBS, etc.).

Ressources

Éditeur en ligne : `https://hydra.ojack.xyz/`. Documentation : `https://hydra.ojack.xyz/docs/`. Pour le GLSL en général : *The Book of Shaders* de Patricio Gonzalez Vivo, `https://thebookofshaders.com/`.

3.5 Resolume

Éditeur : Resolume B.V. (Pays-Bas), depuis 1997.
Versions : *Avenue* (VJ pur, MIDI/OSC/audio reactivity), *Arena* (ajoute le mapping vidéo, sortie multi-écrans, Art-Net pour l'éclairage).
Plateforme : macOS, Windows.
Modèle : payant, propriétaire.

Paradigme

Resolume **n'est pas du live coding**. C'est un *séquenceur visuel temps réel* : on prépare une bibliothèque de clips vidéo, on les organise dans une grille (colonnes = compositions, lignes = couches), on les déclenche au clavier, à la souris, en MIDI ou en OSC, et on applique des effets en chaîne.

Ce n'est pas une faiblesse mais un autre métier. Resolume représente le standard du VJ « pro » : robustesse, latence prédictible, sortie multi-écrans, mapping de surfaces 3D, intégration broadcast.

Architecture mentale

— **Clips** : fichiers vidéo (MOV, MP4, image sequences), souvent en codec optimisé (DXV).
— **Couches** (*layers*) : superposition verticale, mode de fusion (add, multiply, screen).
— **Colonnes** : déclenchement simultané d'un clip par couche — une « scène ».
— **Decks** : ensembles de compositions.
— **Effets** : par clip, par couche, master.
— **Sources externes** : webcam, capture HDMI (BlackMagic), Spout (Win), Syphon (Mac), NDI, Hydra via navigateur capturé.

Pont avec le live coding

C'est ici que les mondes se touchent. On peut envoyer la sortie de Hydra dans Resolume pour : (1) profiter du mapping/scénographie de Resolume, (2) mixer un live coding visuel avec des clips vidéos préparés. Trois protocoles possibles selon la plateforme :

— **Spout** (Windows) : partage de texture GPU entre applications.
— **Syphon** (macOS) : équivalent Mac.
— **NDI** : protocole réseau, multi-plateforme, latence un peu plus élevée mais portable et même utilisable entre machines.

Hydra peut sortir en NDI via une extension navigateur ou via OBS Studio en relais. Resolume reçoit le flux comme une *source vidéo* parmi les autres.

Forces

Stabilité de scène, latence faible, mapping pro, écosystème de plugins (FreeFrameGL, ISF). Standard de fait dans les clubs et festivals.

Limites

Payant (300 € Avenue, 800 € Arena). Pas de génération algorithmique native — on *joue* avec, on n'*écrit* pas. Sortie figée par les clips préparés.

Ressources

`https://resolume.com/`. Manuel officiel détaillé. Tutoriels VJ par *vjlucas*, *Visual Lab* et la communauté française *French VJ Coalition*.

3.6 Vue d'ensemble

Outil	Famille	Paradigme	Niveau	Intégration
Sonic Pi	Audio LC	Boucles nommées	Débutant	SC en interne
Strudel	Audio LC	Patterns fonctionnels	Débutant–avancé	WebAudio, OSC
SuperCollider	Audio LC	Synthèse client/serveur	Avancé	OSC, MIDI
Hydra	Visuel LC	Chaînage GLSL	Débutant–avancé	NDI/Spout/Syphon
Resolume	VJ logiciel	Clips + effets	Débutant–scène	MIDI/OSC, NDI/Spout

Les flèches d'intégration possibles sont nombreuses. Une chaîne typique de fin de cursus :

> *Strudel* (motif rythmique, envoie OSC) → *SuperCollider* (synthèse) → analyse audio → *Hydra* (visuel) → *Resolume* (mapping de scène).

4 Live coding et VJing : deux gestes pour l'image en direct

Cette section est l'articulation théorique du cours. Il faut maintenant penser ensemble *ce qui sépare et ce qui rapproche* le live coding et le VJing, parce que la pratique contemporaine ne respecte plus la frontière — et c'est tant mieux.

4.1 Brève histoire du VJing

Le VJ (*video jockey*) hérite de plusieurs lignées :

- Les **light shows** psychédéliques des années 1960 (Joshua Light Show pour le Fillmore East, Mark Boyle, Andy Warhol et l'Exploding Plastic Inevitable). Projections d'huile et de couleur sur scène en direct.
- Les **video synthesizers** analogiques des années 1970 : Dan Sandin (*Image Processor*, 1971), Steve Rutt et Bill Etra (*Rutt-Etra Scan Processor*). Inspirations directes pour Hydra.
- Le **cinéma élargi** (*expanded cinema*, Gene Youngblood, 1970), qui théorise déjà l'image temps réel comme art performatif.
- Les **clubs et raves** des années 1980–90, où le VJ devient un métier, avec des outils sur mesure puis des logiciels grand public.

Côté logiciel, jalons : *VDMX* (1999), *Modul8* (2003), *Resolume Avenue* (2003 sous ce nom, après *Resolume 2*), *VVVV* (2002, plus proche d'un environnement de programmation), *TouchDesigner* (2008, hybride), *MadMapper* (2010, spécialisé mapping).

Le VJ « pur » prépare une bibliothèque, vit sur scène avec une interface, mixe des images comme un DJ mixe des disques.

4.2 Deux gestes différents

	VJing (Resolume type)	**Live coding visuel (Hydra type)**
Matériau de base	Clips vidéo préparés	Code écrit en direct
Geste de scène	Déclencher, mixer, modifier paramètres	Écrire, modifier, recharger
Préparation	Très lourde (banque de clips)	Légère (quelques snippets de départ)
Improvisation	Sur l'enchaînement et les effets	Sur le contenu lui-même
Risque scénique	Faible (clips testés)	Élevé (un crash est possible)
Identité visuelle	Donnée par les clips choisis	Émergente, parfois imprévue
Public visible	Interface, parfois cachée	Code projeté
Latence ressentie	Très faible	Faible mais variable selon le navigateur/GPU

Ce ne sont pas deux versions d'une même chose : ce sont **deux esthétiques de la performance**. Le VJ assume un répertoire visuel (comme un DJ assume sa discothèque). Le live codeur assume un *processus* dont le résultat n'est pas connu d'avance.

4.3 Ce que les deux ont en commun

— **Le temps réel.** Tout est calculé/déclenché pendant que ça joue. Pas de rendu en aval.
— **L'audio-réactivité.** Que ce soit une enveloppe FFT envoyée à un effet Resolume ou à un `a.fft[0]` dans Hydra, on cherche à coupler le visuel au son.
— **L'improvisation.** Aucun set n'est jamais figé.
— **La performance comme objet.** Le geste de l'opérateur compte, pas seulement le résultat à l'écran.
— **Une économie d'attention partagée** avec un musicien, un public, une lumière. Personne ne joue seul.

4.4 Zones de recouvrement et hybrides

Aux extrêmes, la différence est claire. Au milieu, c'est plus intéressant :

4.4.1 Hydra utilisé comme outil VJ

De plus en plus de VJ « purs » utilisent Hydra non pas en performant le code, mais en préparant des sketches qu'ils déclenchent comme des clips : Hydra devient une *source* dans un setup VJ. Le code n'est plus projeté ; on l'utilise pour sa puissance générative et sa légèreté GPU.

4.4.2 TouchDesigner et le node-based

TouchDesigner (Derivative, Toronto) brouille la frontière autrement : l'environnement est node-based — on patche des opérateurs visuellement comme dans Max/MSP. C'est de la programmation, mais pas textuelle. On peut écrire du Python à l'intérieur. Beaucoup de live codeurs visuels l'utilisent pour des installations ou des shows où le textuel serait trop lent.

C'est l'équivalent visuel de ce que *Pure Data* ou *Max/MSP* sont pour le son : programmation *visuelle*, pas *live coding* au sens TOPLAP — mais générative, temps réel, scriptable.

4.4.3 Routage entre mondes

La pratique scénique mixte est aujourd'hui standard. Quelques chaînes typiques :

```
Hydra (navigateur)
   --[capture: OBS / Spout / Syphon / NDI]-->
Resolume Arena (mapping et scéno)
   --[HDMI]-->
Vidéoprojecteurs / LED walls
```

```
Strudel (rythme + OSC out)
   --[OSC]--> SuperCollider (synthèse audio)
   --[audio bus]--> analyse FFT
   --[OSC]--> Hydra (audio-réactivité)
```

4.4.4 Live coding dans Resolume ?

Marginal mais existant : on peut écrire des shaders ISF (*Interactive Shader Format*, GLSL avec métadonnées) directement dans Resolume Arena 7+. C'est presque du live coding GLSL, à condition d'avoir le réflexe de *recharger* pendant la performance. Peu de VJ exploitent ce filon, mais c'est un terrain d'expérimentation.

4.5 Audio-réactivité : techniques communes

Le couplage son–image est le sujet partagé. Trois techniques de référence :

4.5.1 Analyse FFT

Décomposition d'un signal audio en bandes de fréquences (transformée de Fourier rapide). En général : 4, 8 ou 16 bandes pour une réactivité de scène. La basse module les éléments lents/grands (kick → scale, position), les hautes les détails fins (charley → texture, rotation).

Dans Hydra : `a.fft[i]`. Dans Resolume : `Audio Analysis` comme source de paramètre dans n'importe quel effet. Dans SuperCollider : `FFT.kr` et `Onsets.kr`.

4.5.2 OSC (Open Sound Control)

Protocole UDP textuel, conçu en 1997 à Berkeley pour remplacer le MIDI dans les contextes où la résolution et la flexibilité du MIDI ne suffisent plus. Format : `/path/to/something <args>`.

Exemple : un live codeur Strudel envoie un trigger sur chaque kick, Hydra le reçoit et change de scène.

```
// dans Strudel
s("bd").osc("/kick")
```

```
// dans Hydra (avec extension OSC)
osc(20, 0.1).out(o0)
oscReceive("/kick", () => { osc(40, 0.2, 1).out(o0) })
```

OSC est aussi le protocole interne entre `sclang` et `scsynth` dans SuperCollider — d'où sa centralité dans l'écosystème.

4.5.3 MIDI

Plus ancien (1983), moins flexible mais universel. Une note ou un *control change* peut piloter un paramètre dans Resolume comme dans Hydra. Très utile pour brancher un contrôleur physique (Akai LPD8, Korg nanoKONTROL, Launchpad) en plus du clavier de l'ordinateur — ce qui rapproche le geste live codeur du geste musicien.

4.6 Synthèse

La distinction live coding / VJing est utile au début pour cadrer la pratique. Elle s'efface ensuite : sur scène, ce qui compte est la **cohérence entre le geste et le rendu**, pas l'étiquette de l'outil.

Une formule à retenir : *le VJ choisit ce qu'il joue, le live codeur choisit ce qu'il invente, et le bon performeur sait quand basculer de l'un à l'autre.*